

OSGi Working Group
OSGi Specification repo for
`org.osgi.service.preferences` Release

1.0.0 SNAPSHOT

September 2026



OSGi[®]

LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>"

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
106	PreferencesService Specification	7
106.1	Introduction	7
106.2	Preferences Interface	9
106.3	Concurrency	11
106.4	PreferencesService Interface	12
106.5	Cleanup	12
106.6	org.osgi.service.prefs	12
106.7	References	23

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

106 PreferencesService Specification

Version 1.1

106.1 Introduction

Many bundles need to save some data persistently - in other words, the data is required to survive the stopping and restarting of the bundle and OSGi Framework. In some cases, the data is specific to a particular user. For example, imagine a bundle that implements some kind of game. User specific persistent data could include things like the user's preferred difficulty level for playing the game. Some data is not specific to a user, which we call *system* data. An example would be a table of high scores for the game.

Bundles which need to persist data in an OSGi environment can use the file system via `org.osgi.framework.BundleContext.getDataFile`. A file system, however, can store only bytes and characters, and provides no direct support for named values and different data types.

A popular class used to address this problem for Java applications is the `java.util.Properties` class. This class allows data to be stored as key/value pairs, called *properties*. For example, a property could have a name `com.acme.fudd` and a value of `elmer`. The `Properties` class has rudimentary support for storage and retrieving with its `load` and `store` methods. The `Properties` class, however, has the following limitations:

- Does not support a naming hierarchy.
- Only supports String property values.
- Does not allow its content to be easily stored in a back-end system.
- Has no user name-space management.

Since the `Properties` class was introduced in Java 1.0, efforts have been undertaken to replace it with a more sophisticated mechanism. One of these efforts is this Preferences Service specification.

106.1.1 Essentials

The focus of this specification is simplicity, not reliable access to stored data. This specification does *not* define a general database service with transactions and atomicity guarantees. Instead, it is optimized to deliver the stored information when needed, but it will return defaults, instead of throwing an exception, when the back-end store is not available. This approach may reduce the reliability of the data, but it makes the service easier to use, and allows for a variety of compact and efficient implementations.

This API is made easier to use by the fact that many bundles can be written to ignore any problems that the Preferences Service may have in accessing the back-end store, if there is one. These bundles will mostly or exclusively use the methods of the Preferences interface which are not declared to throw a `BackingStoreException`.

This service only supports the storage of scalar values and byte arrays. It is not intended for storing large data objects like documents or images. No standard limits are placed on the size of data objects which can be stored, but implementations are expected to be optimized for the handling of small objects.

A hierarchical naming model is supported, in contrast to the flat model of the `Properties` class. A hierarchical model maps naturally to many computing problems. For example, maintaining informa-

tion about the positions of adjustable seats in a car requires information for each seat. In a hierarchy, this information can be modeled as a node per seat.

A potential benefit of the Preferences Service is that it allows user specific preferences data to be kept in a well defined place, so that a user management system could locate it. This benefit could be useful for such operations as cleaning up files when a user is removed from the system, or to allow a user's preferences to be cloned for a new user.

The Preferences Service does *not* provide a mechanism to allow one bundle to access the preferences data of another. If a bundle wishes to allow another bundle to access its preferences data, it can pass a [Preferences](#) or [PreferencesService](#) object to that bundle.

The Preferences Service is not intended to provide configuration management functionality. For information regarding Configuration Management, refer to the ???.

106.1.2 Entities

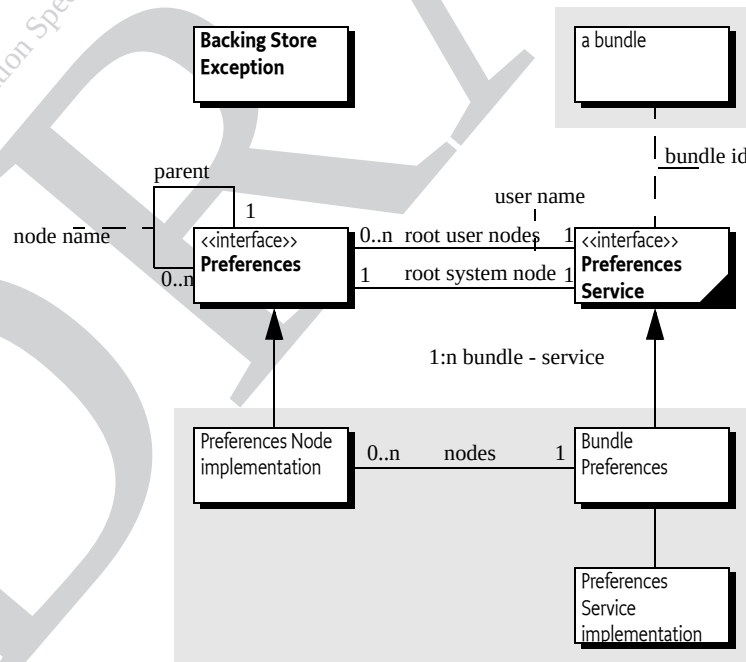
The PreferencesService is a relatively simple service. It provides access to the different roots of Preferences trees. A single system root node and any number of user root nodes are supported. Each *node* of such a tree is an object that implements the Preferences interface.

This Preferences interface provides methods for traversing the tree, as well as methods for accessing the properties of the node. This interface also contains the methods to flush data into persistent storage, and to synchronize the in-memory data cache with the persistent storage.

All nodes except root nodes have a parent. Nodes can have multiple children.

Figure 106.1

Preferences Class Diagram



106.1.3 Operation

The purpose of the Preferences Service specification is to allow bundles to store and retrieve properties stored in a tree of nodes, where each node implements the Preferences interface. The PreferencesService interface allows a bundle to create or obtain a Preferences tree for system properties, as well as a Preferences tree for each user of the bundle.

This specification allows for implementations where the data is stored locally on the Framework or remotely on a back-end system.

106.2 Preferences Interface

Preferences is an interface that defines the methods to manipulate a node and the tree to which it belongs. A Preferences object contains:

- A set of properties in the form of key/value pairs.
- A parent node.
- A number of child nodes.

106.2.1 Hierarchies

A valid Preferences object always belongs to a *tree*. A tree is identified by its root node. In such a tree, a Preferences object always has a single parent, except for a root node which has a null parent.

The root node of a tree can be found by recursively calling the `parent()` method of a node until null is returned. The nodes that are traversed this way are called the *ancestors* of a node.

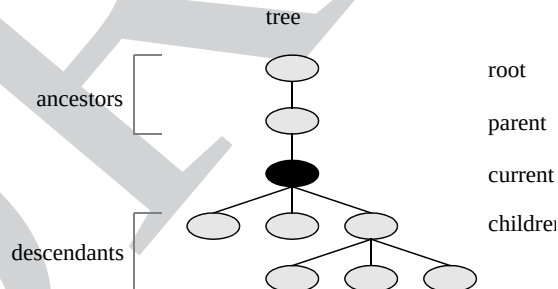
Each Preferences object has a private name-space for child nodes. Each child node has a name that must be unique among its siblings. Child nodes are created by getting a child node with the `node(String)` method. The String argument of this call contains a path name. Path names are explained in the next section.

Child nodes can have child nodes recursively. These objects are called the *descendants* of a node.

Descendants are automatically created when they are obtained from a Preferences object, including any intermediate nodes that are necessary for the given path. If this automatic creation is not desired, the `nodeExists(String)` method can be used to determine if a node already exists.

Figure 106.2

Categorization of nodes in a tree



106.2.2 Naming

Each node has a name relative to its parent. A name may consist of Unicode characters except for the solidus ('/' \u002F). There are no special names, like `..` or `..`.

Empty names are reserved for root nodes. Node names that are directly created by a bundle must *always* contain at least one character.

Preferences node names and property keys are *case sensitive*: for example, `"org.osgi"` and `"oRg.oSgi"` are two distinct names.

The Preferences Service supports different roots, so there is no absolute root for the Preferences Service. This concept is similar to the Windows Registry that also supports a number of roots.

A path consists of one or more node names, separated by a solidus ('/' \u002F). Paths beginning with a solidus ('/' \u002F) are called *absolute paths* while other paths are called *relative paths*. Paths cannot

end with a solidus ('/' \u002F) except for the special case of the root node which has absolute path `"/`.

Path names are always associated with a specific node; this node is called the current node in the following descriptions. Paths identify nodes as follows.

- *Absolute path* - The first `"/` is removed from the path, and the remainder of the path is interpreted as a relative path from the tree's root node.
- *Relative path* -
 - If the path is the empty string, it identifies the current node.
 - If the path is a name (does not contain a `"/`), then it identifies the child node with that name.
 - Otherwise, the first name from the path identifies a child of the current node. The name and solidus ('/' \u002F) are then removed from the path, and the remainder of the path is interpreted as a relative path from the child node.

106.2.3 Tree Traversal Methods

A tree can be traversed and modified with the following methods:

- `childrenNames()` - Returns the names of the child nodes.
- `parent()` - Returns the parent node.
- `removeNode()` - Removes this node and all its descendants.
- `node(String)` - Returns a Preferences object, which is created if it does not already exist. The parameter is an absolute or relative path.
- `nodeExists(String)` - Returns true if the Preferences object identified by the path parameter exists.

106.2.4 Properties

Each Preferences node has a set of key/value pairs called properties. These properties consist of:

- *Key* - A key is a String object and *case sensitive*.
- The name-space of these keys is separate from that of the child nodes. A Preferences node could have both a child node named `fudd` and a property named `fudd`.
- *Value* - A value can always be stored and retrieved as a String object. Therefore, it must be possible to encode/decode all values into/from String objects (though it is not required to store them as such, an implementation is free to store and retrieve the value in any possible way as long as the String semantics are maintained). A number of methods are available to store and retrieve values as primitive types. These methods are provided both for the convenience of the user of the Preferences interface, and to allow an implementation the option of storing the values in a more compact form.

All the keys that are defined in a Preferences object can be obtained with the `keys()` method. The `clear()` method can be used to clear all properties from a Preferences object. A single property can be removed with the `remove(String)` method.

106.2.5 Storing and Retrieving Properties

The Preferences interface has a number of methods for storing and retrieving property values based on their key. All the `put*` methods take as parameters a key and a value. All the `get*` methods take as parameters a key and a default value.

- `put(String,String)`, `get(String,String)`
- `putBoolean(String,boolean)`, `getBoolean(String,boolean)`
- `putInt(String,int)`, `getInt(String,int)`
- `putLong(String,long)`, `getLong(String,long)`

- `putFloat(String,float)` , `getFloat(String,float)`
- `putDouble(String,double)` , `getDouble(String,double)`
- `putByteArray(String,byte[])` , `getByteArray(String,byte[])`

The methods act as if all the values are stored as `String` objects, even though implementations may use different representations for the different types. For example, a property can be written as a `String` object and read back as a float, providing that the string can be parsed as a valid Java float object. In the event of a parsing error, the `get*` methods do not raise exceptions, but instead return their default parameters.

106.2.6

Defaults

All `get*` methods take a default value as a parameter. The reasons for having such a default are:

- When a property for a Preferences object has not been set, the default is returned instead. In most cases, the bundle developer does not have to distinguish whether or not a property exists.
- A *best effort* strategy has been a specific design choice for this specification. The bundle developer should not have to react when the back-end store is not available. In those cases, the default value is returned without further notice.

Bundle developers who want to assure that the back-end store is available should call the `flush` or `sync` method. Either of these methods will throw a `BackingStoreException` if the back-end store is not available.

106.3

Concurrency

This specification specifically allows an implementation to modify Preferences objects in a back-end store. If the back-end store is shared by multiple processes, concurrent updates may cause differences between the back-end store and the in-memory Preferences objects.

Bundle developers can partly control this concurrency with the `flush()` and `sync()` method. Both methods operate on a Preferences object.

The `flush` method performs the following actions:

- Stores (makes persistent) any ancestors (including the current node) that do not exist in the persistent store.
- Stores any properties which have been modified in this node since the last time it was flushed.
- Removes from the persistent store any child nodes that were removed from this object since the last time it was flushed.
- Flushes all existing child nodes.

The `sync` method will first flush, and then ensure that any changes that have been made to the current node and its descendants in the back-end store (by some other process) take effect. For example, it could fetch all the descendants into a local cache, or it could clear all the descendants from the cache so that they will be read from the back-end store as required.

If either method fails, a `BackingStoreException` is thrown.

The `flush` or `sync` methods provide no atomicity guarantee. When updates to the same back-end store are done concurrently by two different processes, the result may be that changes made by different processes are intermingled. To avoid this problem, implementations may simply provide a dedicated section (or name-space) in the back-end store for each OSGi environment, so that clashes do not arise, in which case there is no reason for bundle programmers to ever call `sync`.

In cases where `sync` is used, the bundle programmer needs to take into account that changes from different processes may become intermingled, and the level of granularity that can be assumed is

the individual property level. Hence, for example, if two properties need to be kept in lockstep, so that one should not be changed without a corresponding change to the other, consider combining them into a single property, which would then need to be parsed into its two constituent parts.

106.4 PreferencesService Interface

The PreferencesService is obtained from the Framework's service registry in the normal way. Its purpose is to provide access to Preferences root nodes.

A Preferences Service maintains a system root and a number of user roots. User roots are automatically created, if necessary, when they are requested. Roots are maintained on a per bundle basis. For example, a user root called *elmer* in one bundle is distinct from a user root with the same name in another bundle. Also, each bundle has its own system root. Implementations should use a Service-Factory service object to create a separate PreferencesService object for each bundle.

The precise description of *user* and *system* will vary from one bundle to another. The Preference Service only provides a mechanism, the bundle may use this mechanism in any desired way.

The [PreferencesService](#) interface has the following methods to access the system root and user roots:

- [getSystemPreferences\(\)](#) - Return a Preferences object that is the root of the system preferences tree.
- [getUserPreferences\(String\)](#) - Return a Preferences object associated with the user name that is given as argument. If the user does not exist, a new root is created atomically.
- [getUsers\(\)](#) - Return an array of the names of all the users for whom a Preferences tree exists.

106.5 Cleanup

The Preferences Service must listen for bundle uninstall events, and remove all the preferences data for the bundle that is being uninstalled. The Preferences Service must use the bundle id for the association and not the location.

It also must handle the possibility of a bundle getting uninstalled while the Preferences Service is stopped. Therefore, it must check on startup whether preferences data exists for any bundle which is not currently installed. If it does, that data must be removed.

106.6 org.osgi.service.prefs

Preferences Service Package Version 1.1.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.prefs; version="[1.1,2.0)"} 
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.prefs; version="[1.1,1.2)"} 
```

106.6.1 Summary

- `BackingStoreException` - Thrown to indicate that a preferences operation could not complete because of a failure in the backing store, or a failure to contact the backing store.
- `Preferences` - A node in a hierarchical collection of preference data.
- `PreferencesService` - The Preferences Service.

106.6.2 public class BackingStoreException extends Exception

Thrown to indicate that a preferences operation could not complete because of a failure in the backing store, or a failure to contact the backing store.

106.6.2.1 public BackingStoreException(String message)

message The detail message.

- Constructs a `{@code BackingStoreException}` with the specified detail message.

106.6.2.2 public BackingStoreException(String message, Throwable cause)

message The detail message.

cause The cause of the exception. May be `{@code null}`.

- Constructs a `{@code BackingStoreException}` with the specified detail message.

Since @since 1.1

106.6.2.3 public Throwable getCause()

- Returns the cause of this exception or `{@code null}` if no cause was set.

Returns The cause of this exception or `{@code null}` if no cause was set.

Since @since 1.1

106.6.2.4 public Throwable initCause(Throwable cause)

cause The cause of this exception.

- Initializes the cause of this exception to the specified value.

Returns This exception.

Throws `IllegalArgumentException`— If the specified cause is this exception.

`IllegalStateException`— If the cause of this exception has already been set.

Since @since 1.1

106.6.3 public abstract Preferences

A node in a hierarchical collection of preference data.

This interface allows applications to store and retrieve user and system preference data. This data is stored persistently in an implementation-dependent backing store. Typical implementations include flat files, OS-specific registries, directory servers and SQL databases.

For each bundle, there is a separate tree of nodes for each user, and one for system preferences. The precise description of "user" and "system" will vary from one bundle to another. Typical information stored in the user preference tree might include font choice, and color choice for a bundle which interacts with the user via a servlet. Typical information stored in the system preference tree might include installation data, or things like high score information for a game program.

Nodes in a preference tree are named in a similar fashion to directories in a hierarchical file system. Every node in a preference tree has a *node name* (which is not necessarily unique), a unique *absolute path name*, and a path name *relative* to each ancestor including itself.

The root node has a node name of the empty {@code String} object (`""`). Every other node has an arbitrary node name, specified at the time it is created. The only restrictions on this name are that it cannot be the empty string, and it cannot contain the slash character (`/`).

The root node has an absolute path name of {@code "/"}. Children of the root node have absolute path names of {@code "/" + <node name>}. All other nodes have absolute path names of *<parent's absolute path name> + "/" + <node name>*. Note that all absolute path names begin with the slash character.

A node *n*'s path name relative to its ancestor *a* is simply the string that must be appended to *a*'s absolute path name in order to form *n*'s absolute path name, with the initial slash character (if present) removed. Note that:

- No relative path names begin with the slash character.
- Every node's path name relative to itself is the empty string.
- Every node's path name relative to its parent is its node name (except for the root node, which does not have a parent).
- Every node's path name relative to the root is its absolute path name with the initial slash character removed.

Note finally that:

- No path name contains multiple consecutive slash characters.
- No path name with the exception of the root's absolute path name end in the slash character.
- Any string that conforms to these two rules is a valid path name.

Each {@code Preference} node has zero or more properties associated with it, where a property consists of a name and a value. The bundle writer is free to choose any appropriate names for properties. Their values can be of type {@code String}, {@code long}, {@code int}, {@code boolean}, {@code byte[]}, {@code float}, or {@code double} but they can always be accessed as if they were {@code String} objects.

All node name and property name comparisons are case-sensitive.

All of the methods that modify preference data are permitted to operate asynchronously; they may return immediately, and changes will eventually propagate to the persistent backing store, with an implementation-dependent delay. The {@code flush} method may be used to synchronously force updates to the backing store.

Implementations must automatically attempt to flush to the backing store any pending updates for a bundle's preferences when the bundle is stopped or otherwise ungets the Preferences Service.

The methods in this class may be invoked concurrently by multiple threads in a single Java Virtual Machine (JVM) without the need for external synchronization, and the results will be equivalent to some serial execution. If this class is used concurrently *by multiple JVMs* that store their preference data in the same backing store, the data store will not be corrupted, but no other guarantees are made concerning the consistency of the preference data.

No Implement Consumers of this API must not implement this interface

106.6.3.1

public abstract String absolutePath()

- Returns this node's absolute path name. Note that:
 - Root node - The path name of the root node is {@code "/"}
 - Slash at end - Path names other than that of the root node may not end in slash ({@code '/'}).

- Unusual names - {@code "."} and {@code ".."} have *no* special significance in path names.
- Illegal names - The only illegal path names are those that contain multiple consecutive slashes, or that end in slash and are not the root.

Returns this node's absolute path name.

106.6.3.2 **public abstract String[][] childrenNames() throws BackingStoreException**

- Returns the names of the children of this node. (The returned array will be of size zero if this node has no children and not {@code null}!)

Returns the names of the children of this node.

Throws BackingStoreException— if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

IllegalStateException— if this node (or an ancestor) has been removed with the {@code #removeNode()} method.

106.6.3.3 **public abstract void clear() throws BackingStoreException**

- Removes all of the properties (key-value associations) in this node. This call has no effect on any descendants of this node.

Throws BackingStoreException— if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

IllegalStateException— if this node (or an ancestor) has been removed with the {@code #removeNode()} method.

See Also remove(String)

106.6.3.4 **public abstract void flush() throws BackingStoreException**

- Forces any changes in the contents of this node and its descendants to the persistent store.

Once this method returns successfully, it is safe to assume that all changes made in the subtree rooted at this node prior to the method invocation have become permanent.

Implementations are free to flush changes into the persistent store at any time. They do not need to wait for this method to be called.

When a flush occurs on a newly created node, it is made persistent, as are any ancestors (and descendants) that have yet to be made persistent. Note however that any properties value changes in ancestors are *not* guaranteed to be made persistent.

Throws BackingStoreException— if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

IllegalStateException— if this node (or an ancestor) has been removed with the {@code #removeNode()} method.

See Also sync()

106.6.3.5 **public abstract String get(String key, String def)**

key key whose associated value is to be returned.

def the value to be returned in the event that this node has no value associated with {@code key} or the backing store is inaccessible.

- Returns the value associated with the specified {@code key} in this node. Returns the specified default if there is no value associated with the {@code key}, or the backing store is inaccessible.

Returns the value associated with {@code key}, or {@code def} if no value is associated with {@code key}.

Throws `IllegalStateException`—if this node (or an ancestor) has been removed with the {@link #removeNode()} method.

`NullPointerException`—if {@code key} is {@code null}. (A {@code null} default is permitted.)

106.6.3.6 **public abstract boolean getBoolean(String key, boolean def)**

key {@code key} whose associated value is to be returned as a {@code boolean}.

def the value to be returned in the event that this node has no value associated with {@code key} or the associated value cannot be interpreted as a {@code boolean} or the backing store is inaccessible.

- Returns the {@code boolean} value represented by the {@code String} object associated with the specified {@code key} in this node. Valid strings are "true", which represents {@code true}, and "false", which represents {@code false}. Case is ignored, so, for example, "TRUE" and "False" are also valid. This method is intended for use in conjunction with the {@link #putBoolean(String, boolean)} method.

Returns the specified default if there is no value associated with the {@code key}, the backing store is inaccessible, or if the associated value is something other than "true" or "false", ignoring case.

Returns the {@code boolean} value represented by the {@code String} object associated with {@code key} in this node, or {@code null} if the associated value does not exist or cannot be interpreted as a {@code boolean}.

Throws `NullPointerException`—if {@code key} is {@code null}.

`IllegalStateException`—if this node (or an ancestor) has been removed with the {@link #removeNode()} method.

See Also `get(String,String)`, `putBoolean(String,boolean)`

106.6.3.7 **public abstract byte[][] getByteArray(String key, byte[][] def)**

key {@code key} whose associated value is to be returned as a {@code byte[]} object.

def the value to be returned in the event that this node has no value associated with {@code key} or the associated value cannot be interpreted as a {@code byte[]} type, or the backing store is inaccessible.

- Returns the {@code byte[]} value represented by the {@code String} object associated with the specified {@code key} in this node. Valid {@code String} objects are *Base64* encoded binary data, as defined in RFC 2045 [<http://www.ietf.org/rfc/rfc2045.txt>], Section 6.8, with one minor change: the string must consist solely of characters from the *Base64 Alphabet*; no newline characters or extraneous characters are permitted. This method is intended for use in conjunction with the {@link #putByteArray(String, byte[])} method.

Returns the specified default if there is no value associated with the {@code key}, the backing store is inaccessible, or if the associated value is not a valid Base64 encoded byte array (as defined above).

Returns the {@code byte[]} value represented by the {@code String} object associated with {@code key} in this node, or {@code def} if the associated value does not exist or cannot be interpreted as a {@code byte[]}.

Throws `NullPointerException`—if {@code key} is {@code null}. (A {@code null} value for {@code def} is permitted.)

`IllegalStateException`—if this node (or an ancestor) has been removed with the {@link #removeNode()} method.

See Also `get(String,String)`, `putByteArray(String,byte[])`

106.6.3.8 **public abstract double getDouble(String key, double def)**

key {@code key} whose associated value is to be returned as a {@code double} value.

def the value to be returned in the event that this node has no value associated with {@code key} or the associated value cannot be interpreted as a {@code double} type or the backing store is inaccessible.

- Returns the {@code double} value represented by the {@code String} object associated with the specified {@code key} in this node. The {@code String} object is converted to a {@code double} value as by {@code Double.parseDouble(String)}. Returns the specified default if there is no value associated with the {@code key}, the backing store is inaccessible, or if {@code Double.parseDouble(String)} would throw a {@code NumberFormatException} if the associated value were passed. This method is intended for use in conjunction with the {@code #putDouble} method.

Returns the {@code double} value represented by the {@code String} object associated with {@code key} in this node, or {@code def} if the associated value does not exist or cannot be interpreted as a {@code double} type.

Throws `IllegalStateException`— if this node (or an ancestor) has been removed with the {@code #removeNode()} method.

`NullPointerException`— if {@code key} is {@code null}.

See Also `putDouble(String,double)`, `get(String,String)`

106.6.3.9 **public abstract float getFloat(String key, float def)**

key {@code key} whose associated value is to be returned as a {@code float} value.

def the value to be returned in the event that this node has no value associated with {@code key} or the associated value cannot be interpreted as a {@code float} type or the backing store is inaccessible.

- Returns the float {@code value} represented by the {@code String} object associated with the specified {@code key} in this node. The {@code String} object is converted to a {@code float} value as by {@code Float.parseFloat(String)}. Returns the specified default if there is no value associated with the {@code key}, the backing store is inaccessible, or if {@code Float.parseFloat(String)} would throw a {@code NumberFormatException} if the associated value were passed. This method is intended for use in conjunction with the {@code #putFloat(String, float)} method.

Returns the {@code float} value represented by the string associated with {@code key} in this node, or {@code def} if the associated value does not exist or cannot be interpreted as a {@code float} type.

Throws `IllegalStateException`— if this node (or an ancestor) has been removed with the {@code #removeNode()} method.

`NullPointerException`— if {@code key} is {@code null}.

See Also `putFloat(String,float)`, `get(String,String)`

106.6.3.10 **public abstract int getInt(String key, int def)**

key key whose associated value is to be returned as an {@code int}.

def the value to be returned in the event that this node has no value associated with {@code key} or the associated value cannot be interpreted as an {@code int} or the backing store is inaccessible.

- Returns the {@code int} value represented by the {@code String} object associated with the specified {@code key} in this node. The {@code String} object is converted to an {@code int} as by {@code Integer.parseInt(String)}. Returns the specified default if there is no value associated with the {@code key}, the backing store is inaccessible, or if {@code Integer.parseInt(String)} would throw a {@code NumberFormatException} if the associated {@code value} were passed. This method is intended for use in conjunction with the {@code #putInt(String, int)} method.

Returns the {@code int} value represented by the {@code String} object associated with {@code key} in this node, or {@code def} if the associated value does not exist or cannot be interpreted as an {@code int} type.

Throws `NullPointerException`— if {@code key} is {@code null}.

`IllegalStateException`— if this node (or an ancestor) has been removed with the `{@link #removeNode()}` method.

See Also `putInt(String,int)`, `get(String,String)`

106.6.3.11 **public abstract long getLong(String key, long def)**

key `{@code key}` whose associated value is to be returned as a `{@code long}` value.

def the value to be returned in the event that this node has no value associated with `{@code key}` or the associated value cannot be interpreted as a `{@code long}` type or the backing store is inaccessible.

- Returns the `{@code long}` value represented by the `{@code String}` object associated with the specified `{@code key}` in this node. The `{@code String}` object is converted to a `{@code long}` as by `{@code Long.parseLong(String)}`. Returns the specified default if there is no value associated with the `{@code key}`, the backing store is inaccessible, or if `{@code Long.parseLong(String)}` would throw a `{@code NumberFormatException}` if the associated `{@code value}` were passed. This method is intended for use in conjunction with the `{@link #putLong(String, long)}` method.

Returns the `{@code long}` value represented by the `{@code String}` object associated with `{@code key}` in this node, or `{@code def}` if the associated value does not exist or cannot be interpreted as a `{@code long}` type.

Throws `NullPointerException`— if `{@code key}` is `{@code null}`.

`IllegalStateException`— if this node (or an ancestor) has been removed with the `{@link #removeNode()}` method.

See Also `putLong(String,long)`, `get(String,String)`

106.6.3.12 **public abstract String[][] keys() throws BackingStoreException**

- Returns all of the keys that have an associated value in this node. (The returned array will be of size zero if this node has no preferences and not `{@code null}`!)

Returns an array of the keys that have an associated value in this node.

Throws `BackingStoreException`— if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

`IllegalStateException`— if this node (or an ancestor) has been removed with the `{@link #removeNode()}` method.

106.6.3.13 **public abstract String name()**

- Returns this node's name, relative to its parent.

Returns this node's name, relative to its parent.

106.6.3.14 **public abstract Preferences node(String pathName)**

pathName the path name of the `{@code Preferences}` object to return.

- Returns a named `{@code Preferences}` object (node), creating it and any of its ancestors if they do not already exist. Accepts a relative or absolute pathname. Absolute pathnames (which begin with `{@code '/'}`) are interpreted relative to the root of this node. Relative pathnames (which begin with any character other than `{@code '/'}`) are interpreted relative to this node itself. The empty string (`{@code ""}`) is a valid relative pathname, referring to this node itself.

If the returned node did not exist prior to this call, this node and any ancestors that were created by this call are not guaranteed to become persistent until the `{@code flush}` method is called on the returned node (or one of its descendants).

Returns the specified `{@code Preferences}` object.

Throws `IllegalArgumentException`— if the path name is invalid.

`IllegalStateException`—if this node (or an ancestor) has been removed with the {@link #removeNode()} method.

`NullPointerException`—if path name is {@code null}.

See Also `flush()`

106.6.3.15 **public abstract boolean nodeExists(String pathName) throws BackingStoreException**

pathName the path name of the node whose existence is to be checked.

- Returns true if the named node exists. Accepts a relative or absolute pathname. Absolute pathnames (which begin with {@code '/'}) are interpreted relative to the root of this node. Relative pathnames (which begin with any character other than {@code '/'}) are interpreted relative to this node itself. The pathname {@code ""} is valid, and refers to this node itself.

If this node (or an ancestor) has already been removed with the {@link #removeNode()} method, it is legal to invoke this method, but only with the pathname {@code ""}; the invocation will return {@code false}. Thus, the idiom {@code p.nodeExists("")} may be used to test whether {@code p} has been removed.

Returns true if the specified node exists.

Throws `BackingStoreException`—if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

`IllegalStateException`—if this node (or an ancestor) has been removed with the {@link #removeNode()} method and {@code pathname} is not the empty string ({@code ""}).

`IllegalArgumentException`—if the path name is invalid (i.e., it contains multiple consecutive slash characters, or ends with a slash character and is more than one character long).

106.6.3.16 **public abstract Preferences parent()**

- Returns the parent of this node, or {@code null} if this is the root.

Returns the parent of this node.

Throws `IllegalStateException`—if this node (or an ancestor) has been removed with the {@link #removeNode()} method.

106.6.3.17 **public abstract void put(String key, String value)**

key key with which the specified value is to be associated.

value value to be associated with the specified key.

- Associates the specified value with the specified key in this node.

Throws `NullPointerException`—if {@code key} or {@code value} is {@code null}.

`IllegalStateException`—if this node (or an ancestor) has been removed with the {@link #removeNode()} method.

106.6.3.18 **public abstract void putBoolean(String key, boolean value)**

key {@code key} with which the string form of value is to be associated.

value value whose string form is to be associated with {@code key}.

- Associates a {@code String} object representing the specified {@code boolean} value with the specified key in this node. The associated string is "true" if the value is {@code true}, and "false" if it is {@code false}. This method is intended for use in conjunction with the {@link #getBoolean(String, boolean)} method.

Implementor's note: it is *not* necessary that the value be represented by a string in the backing store. If the backing store supports {@code boolean} values, it is not unreasonable to use them. This im-

plementation detail is not visible through the {@code Preferences} API, which allows the value to be read as a {@code boolean} (with {@code getBoolean}) or a {@code String} (with {@code get}) type.

Throws `NullPointerException` – if {@code key} is {@code null}.

`IllegalStateException` – if this node (or an ancestor) has been removed with the {@code removeNode()} method.

See Also `getBoolean(String,boolean)`, `get(String,String)`

106.6.3.19 **public abstract void putByteArray(String key, byte[][] value)**

key {@code key} with which the string form of {@code value} is to be associated.

value {@code value} whose string form is to be associated with {@code key}.

- Associates a {@code String} object representing the specified {@code byte[]} with the specified {@code key} in this node. The associated {@code String} object is the *Base64* encoding of the {@code byte[]}, as defined in RFC 2045 [http://www.ietf.org/rfc/rfc2045.txt], Section 6.8, with one minor change: the string will consist solely of characters from the *Base64 Alphabet*; it will not contain any newline characters. This method is intended for use in conjunction with the {@code getByteArray(String, byte[])} method.

Implementor's note: it is *not* necessary that the value be represented by a {@code String} type in the backing store. If the backing store supports {@code byte[]} values, it is not unreasonable to use them. This implementation detail is not visible through the {@code Preferences} API, which allows the value to be read as a {@code byte[]} object (with {@code getByteArray}) or a {@code String} object (with {@code get}).

Throws `NullPointerException` – if {@code key} or {@code value} is {@code null}.

`IllegalStateException` – if this node (or an ancestor) has been removed with the {@code removeNode()} method.

See Also `getByteArray(String,byte[])`, `get(String,String)`

106.6.3.20 **public abstract void putDouble(String key, double value)**

key {@code key} with which the string form of value is to be associated.

value value whose string form is to be associated with {@code key}.

- Associates a {@code String} object representing the specified {@code double} value with the specified {@code key} in this node. The associated {@code String} object is the one that would be returned if the {@code double} value were passed to {@code Double.toString(double)}. This method is intended for use in conjunction with the {@code getDouble(String, double)} method

Implementor's note: it is *not* necessary that the value be represented by a string in the backing store. If the backing store supports {@code double} values, it is not unreasonable to use them. This implementation detail is not visible through the {@code Preferences} API, which allows the value to be read as a {@code double} (with {@code getDouble}) or a {@code String} (with {@code get}) type.

Throws `NullPointerException` – if {@code key} is {@code null}.

`IllegalStateException` – if this node (or an ancestor) has been removed with the {@code removeNode()} method.

See Also `getDouble(String,double)`

106.6.3.21 **public abstract void putFloat(String key, float value)**

key {@code key} with which the string form of value is to be associated.

value value whose string form is to be associated with {@code key}.

- Associates a {@code String} object representing the specified {@code float} value with the specified {@code key} in this node. The associated {@code String} object is the one that would be returned if the {@code float} value were passed to {@code Float.toString(float)}. This method is intended for use in conjunction with the {@code #getFloat(String, float)} method.

Implementor's note: it is *not* necessary that the value be represented by a string in the backing store. If the backing store supports {@code float} values, it is not unreasonable to use them. This implementation detail is not visible through the {@code Preferences} API, which allows the value to be read as a {@code float} (with {@code getFloat}) or a {@code String} (with {@code get}) type.

Throws `NullPointerException` – if {@code key} is {@code null}.

`IllegalStateException` – if this node (or an ancestor) has been removed with the {@code #removeNode()} method.

See Also `getFloat(String, float)`

106.6.3.22 **public abstract void putInt(String key, int value)**

key key with which the string form of value is to be associated.

value {@code value} whose string form is to be associated with {@code key}.

- Associates a {@code String} object representing the specified {@code int} value with the specified {@code key} in this node. The associated string is the one that would be returned if the {@code int} value were passed to {@code Integer.toString(int)}. This method is intended for use in conjunction with {@code #getInt(String, int)} method.

Implementor's note: it is *not* necessary that the property value be represented by a {@code String} object in the backing store. If the backing store supports integer values, it is not unreasonable to use them. This implementation detail is not visible through the {@code Preferences} API, which allows the value to be read as an {@code int} (with {@code getInt}) or a {@code String} (with {@code get}) type.

Throws `NullPointerException` – if {@code key} is {@code null}.

`IllegalStateException` – if this node (or an ancestor) has been removed with the {@code #removeNode()} method.

See Also `getInt(String, int)`

106.6.3.23 **public abstract void putLong(String key, long value)**

key {@code key} with which the string form of {@code value} is to be associated.

value {@code value} whose string form is to be associated with {@code key}.

- Associates a {@code String} object representing the specified {@code long} value with the specified {@code key} in this node. The associated {@code String} object is the one that would be returned if the {@code long} value were passed to {@code Long.toString(long)}. This method is intended for use in conjunction with the {@code #getLong(String, long)} method.

Implementor's note: it is *not* necessary that the {@code value} be represented by a {@code String} type in the backing store. If the backing store supports {@code long} values, it is not unreasonable to use them. This implementation detail is not visible through the {@code Preferences} API, which allows the value to be read as a {@code long} (with {@code getLong}) or a {@code String} (with {@code get}) type.

Throws `NullPointerException` – if {@code key} is {@code null}.

`IllegalStateException` – if this node (or an ancestor) has been removed with the {@code #removeNode()} method.

See Also `getLong(String, long)`

106.6.3.24 public abstract void remove(String key)

key key whose mapping is to be removed from this node.

- Removes the value associated with the specified {@code key} in this node, if any.

Throws `IllegalStateException`— if this node (or an ancestor) has been removed with the {@code removeNode()} method.

See Also `get(String,String)`

106.6.3.25 public abstract void removeNode() throws BackingStoreException

- Removes this node and all of its descendants, invalidating any properties contained in the removed nodes. Once a node has been removed, attempting any method other than {@code name()}, {@code absolutePath()} or {@code nodeExists("")} on the corresponding {@code Preferences} instance will fail with an `IllegalStateException`. (The methods defined on {@code Object} can still be invoked on a node after it has been removed; they will not throw `IllegalStateException`.)

The removal is not guaranteed to be persistent until the `flush` method is called on the parent of this node.

Throws `IllegalStateException`— if this node (or an ancestor) has already been removed with the `removeNode()` method.

`BackingStoreException`— if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

See Also `flush()`

106.6.3.26 public abstract void sync() throws BackingStoreException

- Ensures that future reads from this node and its descendants reflect any changes that were committed to the persistent store (from any VM) prior to the `sync` invocation. As a side-effect, forces any changes in the contents of this node and its descendants to the persistent store, as if the `flush` method had been invoked on this node.

Throws `BackingStoreException`— if this operation cannot be completed due to a failure in the backing store, or inability to communicate with it.

`IllegalStateException`— if this node (or an ancestor) has been removed with the `removeNode()` method.

See Also `flush()`

106.6.4 public abstract PreferencesService

The Preferences Service.

Each bundle using this service has its own set of preference trees: one for system preferences, and one for each user.

A `PreferencesService` object is specific to the bundle which obtained it from the service registry. If a bundle wishes to allow another bundle to access its preferences, it should pass its `PreferencesService` object to that bundle.

No Implement Consumers of this API must not implement this interface

106.6.4.1 public abstract Preferences getSystemPreferences()

- Returns the root system node for the calling bundle.

Returns The root system node for the calling bundle.

106.6.4.2 public abstract Preferences getUserPreferences(String name)

name The user for which to return the preference root node.

- Returns the root node for the specified user and the calling bundle.

Returns The root node for the specified user and the calling bundle.

106.6.4.3 **public abstract String[][] getUsers()**

- Returns the names of users for which node trees exist.

Returns The names of users for which node trees exist.

106.7 References

- [1] *JSR 10 Preferences API*
<https://www.jcp.org/en/jsr/detail?id=10>
- [2] *RFC 2045 Base 64 encoding*
<https://www.ietf.org/rfc/rfc2045.txt>

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT